Stack Data Structure

1. Introduction to Stack

A stack is a linear data structure in which insertion and deletion of elements take place at only one end, called the TOP.

A stack follows the LIFO (Last In, First Out) principle. The element inserted last is removed first.

Real-life examples:

- A stack of plates in a cafeteria.
- Undo and redo operations in a text editor.
- The Back button in a web browser.
- Function calls and recursion in programming.
- Reversing a string.

Visual representation of a stack

TOP

30

20

10

Bottom

Element 30 was inserted last, so it will be removed first.

2. Basic Stack Operations

Operation	**Description**
push(x)	Inserts element x at the top.
pop()	Removes and returns the top element.
peek() / top()	Returns the top element without removing it.
isEmpty()	Checks whether the stack is empty.
isFull()	Checks whether an array-based stack is full.
size()	Returns the number of elements.
display()	Displays the stack elements.

Example of stack operations

Suppose the stack is initially empty.

Operation	**Stack (Bottom → Top)**	**TOP**

push(10)	10	10
push(20)	10, 20	20
push(30)	10, 20, 30	30
pop()	10, 20	20
push(40)	10, 20, 40	40
peek()	10, 20, 40	40

The peek() operation returns 40 without removing it.

3. Stack Overflow and Stack Underflow

- Stack Overflow: Occurs when an element is pushed into a full, fixed-capacity stack.
- Stack Underflow: Occurs when an element is popped from an empty stack.

Example: If an array-based stack has a capacity of 3 and already contains [10, 20, 30], pushing 40 causes overflow. Popping from an empty stack causes underflow.

4. Implementation of Stack

A stack can be implemented using:

1. Arrays or Python lists.
2. Linked lists.

5. Important Applications of Stack

5.1 Function calls and recursion

Programming languages use a call stack to manage function calls. Each call creates a stack frame containing information such as local variables and return addresses.

For example, recursive factorial:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
print(factorial(4))
# 24
```

The calls factorial(4), factorial(3), factorial(2), and factorial(1) are managed through the call stack. The results are returned in reverse order.

5.2 Reversing a string

```
def reverse_string(text):
```

```
stack = []
for ch in text:
        stack.append(ch)
        result = ""
        while stack:
                result += stack.pop()
        return resultprint(reverse_string("MCA"))
```

5.3 Parentheses matching

A stack checks whether brackets are balanced and correctly nested.

Examples:

- {[()]} — Balanced
- [(]) — Not balanced
- ((())) — Balanced
- (() — Not balanced

Algorithm:

1. Scan the expression from left to right.
2. Push each opening bracket onto the stack.
3. For each closing bracket, check whether the stack is empty.
4. If it is empty, the expression is invalid.
5. Otherwise, pop the top bracket and check whether the types match.
6. At the end, the expression is balanced only if the stack is empty.

5.4 Infix, Prefix and Postfix Expressions

Stack is widely used for expression conversion and evaluation.

Notation	Example	Meaning
Infix	A + B	Operator between operands
Prefix	+ A B	Operator before operands
Postfix	A B +	Operator after operands

For the expression (A + B) * C:

- Infix: (A + B) * C
- Prefix: * + A B C

- Postfix: A B + C *

Evaluating a postfix expression

Consider 5 3 + 2 *.

1. Push 5.
2. Push 3.
3. Encounter +: pop 3 and 5, calculate \(5+3=8\), and push 8.
4. Push 2.
5. Encounter *: pop 2 and 8, calculate \(8\times2=16\), and push 16.

Final result: 16.

5.5 Undo and Redo Operations

Text editors use stacks to track previous actions.

- Undo stack: Stores actions that can be reversed.
- Redo stack: Stores actions that were undone and can be reapplied.

When a new action is performed, the redo history is generally cleared.

5.6 Depth-First Search (DFS)

Depth-First Search uses a stack to explore a graph by visiting a vertex and proceeding as deeply as possible before backtracking.

DFS can be implemented using:

- An explicit stack in an iterative algorithm.
- The programming language's call stack through recursion.

Other applications include browser history, backtracking, maze solving, and certain compiler operations.

6. Common Errors and Edge Cases

Situation	**Expected handling**
Pop from an empty stack	Report underflow or return a defined error value
Peek on an empty stack	Report empty stack or return None
Push into a full fixed-size stack	Report overflow
Pop the only element	Set the stack to empty
Search for an absent element	Report that the element was not found